



Chapitre 7: La Programmation Modulaire

Babacar DIOP

Adresse Mail: diopbabacar4888@gmail.com

Téléphone: 777812040

Problématique

❖ **Contexte du problème** : Nous devons écrire un programme qui effectue les opérations suivantes :

- ☐ Remplir un tableau de N entiers saisis par l'utilisateur.
- ☐ Afficher le contenu du tableau initial.
- ☐ Réaliser un décalage cyclique du tableau d'un rang vers la droite.
- ☐ Afficher le tableau après décalage.

❖ **Observation**: L'opération d'affichage est réalisée deux fois dans ce programme : avant et après le décalage. Répéter le même code pour ces deux affichages est contraire aux bonnes pratiques de la programmation, car cela nuit à la lisibilité, à la maintenance et à la réutilisabilité du code



Introduction

- ❖ Un programme informatique est un ensemble structuré d'instructions conçues pour résoudre un ou plusieurs problèmes spécifiques. Très souvent, une seule action logique nécessite plusieurs instructions. Pour améliorer la clarté, la lisibilité et la réutilisabilité du code, il est recommandé de regrouper ces instructions en blocs fonctionnels appelés **modules** ou **sous-programmes**.
- ❖ L'utilisation des sous-programmes présente plusieurs avantages :
 - ❑ Éviter la répétition du code (principe de non-redondance),
 - ❑ Faciliter l'organisation du programme,
 - ❑ Améliorer la lisibilité et la maintenance,
 - ❑ Encourager la réutilisation des blocs de code.
- ❖ En langage C, on distingue principalement deux types de sous-programmes :
 - ❑ Les fonctions, qui effectuent un traitement et retournent une valeur,
 - ❑ Les procédures, qui effectuent un traitement sans retourner de valeur (en C, ce sont aussi des fonctions, mais de type void).
- ❖ Dans ce cours, nous allons apprendre à concevoir des programmes modulaires en utilisant efficacement ces deux types de sous-programmes.

I. Fonction

1. **Définition:** Une fonction est sous-programme qui doit obligatoirement retourner un et un seul résultat.

2. **Syntaxe:**

```
TypeRetour nomFonction ( [type1 param1,..., typen paramn] ){  
    //Corps de la fonction  
    return expression ;  
}
```

NB : Le type de **expression** doit être toujours identique au **type de retour**.

3. **Prototype:** Il représente l'entête de la fonction

```
TypeRetour nomFonction ( [type1 param1,..., typen paramn] )
```

4. **Remarques**

- ❖ Les noms des variables des paramètres sont facultatifs pour les prototypes.
- ❖ Il est impossible de regrouper les paramètres par type
- ❖ Les noms de variables sont obligatoires lors de l'implémentation
- ❖ Il existe deux types de variables avec les sous-programmes :
 - ☐ Les variables globales : elles sont déclarées en dehors de toutes les fonctions et généralement en haut
 - ☐ Les variables locales : elles sont déclarées au sein des fonctions. Elles ont une portée limitée.

II. Procédure

1. **Définition:** Une procédure est une fonction qui ne retourne aucune valeur. Elle est déclarée avec le mot-clé void et s'utilise lorsqu'un bloc d'instructions doit être exécuté sans produire de résultat à utiliser ailleurs dans le programme.

2. **Remarques:**

- ❖ Une procédure commence toujours par void.
- ❖ Elle peut recevoir des paramètres (ou non), selon les besoins.
- ❖ On appelle une procédure en écrivant son nom suivi de parenthèses contenant les éventuels paramètres réels

3. **Exemple:**

```
void afficherBienvenue() {  
    printf("Bienvenue dans le cours de programmation modulaire !\n");  
}  
  
int main() {  
    // Appel de la procédure  
    afficherBienvenue();  
    return 0;  
}
```

- La procédure afficherBienvenue affiche simplement un message à l'écran.
- Elle ne retourne aucune valeur, d'où l'utilisation de void.
- Elle est appelée dans la fonction main sans paramètre.

Exercice d'Application

On vous propose de faire un programme qui, à partir d'un menu, nous permet de faire les actions suivantes :

- ☐ Somme (choix 1)
- ☐ Différence (choix 2)
- ☐ Produit (choix 3)
- ☐ Quotient Entier (Choix 4)
- ☐ Modulo (choix 5)
- ☐ Quitter (choix 6)

NB : Tous les choix (sauf choix 6) seront des modules à définir.

Contraintes :

- Utilisation des structures à choix multiples
- Utilisation des modules
- Le code doit être indenté et commenté